

Algoritmi paraleli de înmulțire a matricelor cu vectori

Obiectivele lucrării

Obiectivul acestui laborator este acela de a dezvolta un program paralel de înmulțire a matricelor cu vectori. Sarcinile laboratorului includ:

- ⇒ Exercițiul 1 – Enunțarea problemei de înmulțire a matricelor cu vectori.
- ⇒ Exercițiul 2 – Codul programului serial ce permite înmulțirea matricelor cu vectori.
- ⇒ Exercițiul 3 – Dezvoltarea algoritmului paralel de înmulțire a matricelor cu vectori.
- ⇒ Exercițiul 4 – Codul programului paralel ce permite înmulțirea matricelor cu vectori.

Se pornește de la premisa că studenții sunt familiarizați cu „Programarea paralelă utilizând standardul MPI”. De asemenea, laboratoarele preliminare „Introducere în programare paralelă și distribuită - standardul MPI” și „Programare paralelă și distribuită în MPI” se presupun a fi încheiate.

Exercițiul 1 – Problema înmulțirii matricelor cu vectori

Rezultatul înmulțirii unei matrice A de dimensiune $m \times n$ cu un vector b , care are n elemente, este un vector c de mărime m . Fiecare element i din vector se obține ca rezultat al înmulțirii scalare dintre rândul i al matricei A (notăm rândul i cu a_i) și vectorul b (figura. 3.1).

$$c_i = (a_i, b) = \sum_{j=1}^n a_{ij} b_j, \quad 1 \leq i \leq m \quad (3.1)$$

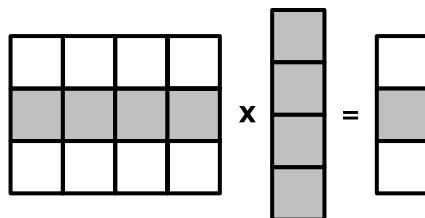


Figura 3.1 Înmulțire matrice-vector

Spre exemplu, dacă matricea compusă din 3 rânduri și 4 coloane este înmulțită cu un vector compus din 4 elemente, vom obține ca rezultat un vector de mărime 3 (figura. 3.2).

$$\begin{pmatrix} 3 & 2 & 0 & -1 \\ 5 & -2 & 1 & 1 \\ 1 & 0 & -1 & -1 \end{pmatrix} \times \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \end{pmatrix} = \begin{pmatrix} 3 \\ 8 \\ -6 \end{pmatrix}$$

Figura 3.2 Exemplu de înmulțire matrice-vector

Așadar, obținerea vectorului c se face executând m operații, de același tip, de înmulțire între rândurile matricei A și vectorul b . Pseudocodul pentru înmulțirea unei matrice cu un vector poate fi următorul:

```
//Algoritm serial de inmultire matrice-vector
for(int i=0;i<m;i++)
{
    c[i] = 0;
    for(j=0;j<n;j++)
        c[i] += A[i][j]*b[j];
}
```

Exercițiul 2 – Codul serial al programului ce permite înmulțirea matricelor cu vectori

În acest exercițiu, vom implementa algoritmul serial de înmulțire a unei matrice cu un vector. Versiunea inițială a programului ce va fi realizat este prezentată în proiectul *SerialMatriceVectorMult*, care conține o parte a codului inițial și asigură parametrii necesari pentru proiect. Următoarele operații urmează să fie adăugate la versiunea inițială a programului: dimensiunea matricei și a vectorului, inițializarea matricei și a vectorului, înmulțirea propriu-zisă a matricei A cu vectorul b , și afișarea rezultatelor.

Pas 1 – Deschiderea proiectului *SerialMatriceVectorMult*

Se va deschide proiectul *SerialMatriceVectorMult* parcurgând următorii pași:

- ⇒ Start Microsoft **Visual Studio 2005**.
- ⇒ Se execută comanda **Open** → **Project/Solution** în meniul **File**.
- ⇒ Se alege directorul *SerialMatriceVectorMult* în fereastra **Open Project**.

⇒ Se execută dublu click pe fișierul **SerialMatriceVectorMult.sln**.

După ce s-a deschis proiectul, în fereastra **Solution Explorer** (Ctrl+Alt+L), se execută dublu click pe fișierul **SerialMV.cpp**, ca în figura 3.3. Apoi, codul ce urmează a fi îmbunătățit, va fi deschis în fereastra **Visual Studio**.

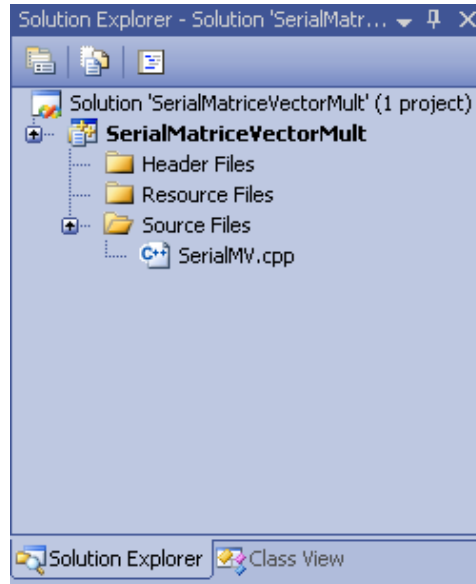


Figura 3.3 Deschiderea fișierului *SerialMV.cpp*

Fișierul **SerialMV.cpp** permite accesul la librăriile necesare și conține, de asemenea, variabilele necesare pentru dezvoltarea aplicației, precum și mesajele ce trebuie afișate către utilizatori.

```
double* pMatrice; //Matricea initiala
double* pVector; //Vectorul initial
double* pRezultat; //Vectorul in care se va depune
rezultatul inmultirii
int Dim; // Dimensiunea matricii si a vectorului initial
```

Primele două variabile (**pMatrice** și **pVector**) corespund matricii și vectorului ce participă ca argumente în funcția de înmulțire. A treia variabilă, **pRezultat**, reprezintă vectorul ce va fi obținut în urma înmulțirii dintre matricea **pMatrice** și vectorul **pVector**. Variabila **Dim** determină dimensiunea matricii și a vectorului (matricea **pMatrice** este pătratică și are dimensiunea **Dim * Dim**). De notat este faptul că matricea este memorată

într-un câmp uni-dimensional. Astfel, elementul găsit la intersecția rândului i și coloanei j a matricei are indexul $i*Dim+j$ în câmpul uni-dimensional.

Codul de program ce urmează după declarația variabilelor este un mesaj de întâmpinare și așteptare pentru apăsarea unei taste înainte de părăsirea aplicației.

```
printf("Program pentru inmultirea seriala a matricelor cu\nvectori\n");  
getch();
```

Pentru rularea aplicației se selectează comanda **Rebuild Solution** din meniul **Build**. Dacă aplicația este compilată cu succes (în partea de jos a ferestrei **Visual Studio** apare „*Rebuild All: 1 succeeded, 0 failed, 0 skipped*”), se apasă tasta **F5** sau se execută comanda **Start Debugging** din meniul **Debug**.

Pas 2 – Citirea dimensiunii matricei și a vectorului de intrare

Pentru inițializarea datelor din acest program am implementat funcția *InitializareProces*. Aceasta funcție determină dimensiunea obiectelor, alocă memoria necesară pentru realizarea operației de înmulțire (matricea inițială *pMatrice* și vectorul *pVector*, și vectorul rezultat *pRezultat*). De asemenea, tot în aceasta funcție se setează și valorile inițiale ale vectorului și ale matricei. Astfel, funcția va avea următorul antet:

```
// Functie ce alocă memoria necesară și inițializează  
matricea și vectorul  
void InitializareProces (double* &pMatrice, double*  
&pVector, double* &pRezultat, int &Dim)
```

În prima etapă este necesar să determinăm dimensiunea obiectelor (prin setarea valorii variabilei *Dim*). Codul de mai jos va trebui adăugat în funcția *InitializareProces*:

```
void InitializareProces (double* &pMatrice, double*  
&pVector, double* &pRezultat, int &Dim)  
{  
    printf("\nIntroduceti dimensiunea obiectelor: ");  
    scanf("%d", &Dim);  
    printf("\nDimensiunea obiectelor este %d\n", Dim);  
}
```

Astfel, utilizatorul poate seta dimensiunea obiectelor de intrare (dimensiunea matricei și a vectorului), ce vor fi ulterior citite de program – folosind stream-ul de intrare *stdin* – și memorate în variabila *Dim*. Apoi valoarea variabilei *Dim* este afișată pe ecran (figura 3.4). Funcția *InitializareProces* va trebui apelată din funcția *main* imediat după afișarea mesajului inițial.

```
//Programul principal
void main() {
    double* pMatrice; // Matricea initiala
    double* pVector; // Vectorul initial
    double* pRezultat; // Vectorul rezultat inmultire
    int Dim; // Dimensiunea obiectelor

    printf("Program pentru inmultirea seriala a matricelor
cu vectori\n");

    // Alocare memorie si initializare
    InitializareProces(pMatrice, pVector, pRezultat, Dim);
    getch();
}
```

Se compilează și se rulează aplicația, astfel asigurându-ne că valoarea memorată în variabila *Dim* este cea corectă.

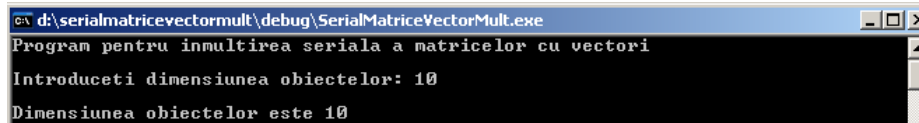


Figura 3.4 Setarea dimensiunii obiectelor

În continuare aplicația trebuie să testeze dacă utilizatorul a introdus o valoare corectă pentru dimensiunea matricei și a vectorului. Fragmentul de cod ce verifică acest lucru este următorul:

```
// Setarea dimensiunii matricii si vectorului
do {
    printf("\nIntroduceti dimensiunea obiectelor: ");
    scanf("%d", &Dim);
    printf("\nDimensiunea obiectelor este %d\n", Dim);
    if (Dim <= 0)
        printf("\nDimensiunea trebuie sa fie mai mare
decat 0!\n");
}while (Dim <= 0);
```

Compilați și rulați aplicația. Asigurați-vă că situațiile incorecte sunt procesate corect în aplicație.

Pas 3 – Citirea datelor de intrare

Funcția de inițializare trebuie, de asemenea, să permită alocarea memoriei necesare pentru stocarea obiectelor (adăugați codul de mai jos în funcția *InitializareProces*):

```
// Functie ce alocă memoria necesară și inițializează  
matricea și vectorul  
void InitializareProces (double* &pMatrice, double*  
&pVector, double* &pRezultat, int &Dim)  
{  
    // Setarea dimensiunii matricii și vectorului  
    do {  
        <...>  
    }while (Dim <= 0);  
    // Alocare memorie  
    pMatrice = new double [Dim*Dim];  
    pVector = new double [Dim];  
    pRezultat = new double [Dim];  
}
```

În continuare este necesar să setăm valorile inițiale ale matricii *pMatrice* și ale vectorului *pVector*. Pentru acest scop, va trebui construită funcția *InitializareSimpla*:

```
// Functie ce permite inițializarea rapidă a elementelor  
matricii și vectorului  
void InitializareSimpla (double* pMatrice, double* pVector,  
int Dim)  
{  
    int i, j; // variabile necesare în buclă  
    for (i=0; i<Dim; i++)  
    {  
        pVector[i] = 1;  
        for (j=0; j<Dim; j++)  
            pMatrice[i*Dim+j] = i;  
    }  
}
```

După cum rezultă și din codul de mai sus, această funcție setează elementele matricii și ale vectorului în cel mai simplu mod posibil: valoarea elementelor matricii corespunde cu numărul rândului în care sunt alocate și toate elementele vectorului au valoarea 1. În cazul în care utilizatorul alege dimensiunea egală cu 4, matricea și vectorul vor fi următoarele:

$$pMatrice = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 2 & 2 & 2 & 2 \\ 3 & 3 & 3 & 3 \end{pmatrix}, \quad pVector = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix}$$

Setarea aleatoare a datelor va fi discutată la Pasul 6. Funcția *InitializareSimpla* va fi apelată în funcția *InitializareProces* imediat după alocarea memoriei necesare.

```
// Functie ce alocă memoria necesară și initializează  
matricea și vectorul  
void InitializareProces (double* &pMatrice, double*  
&pVector, double* &pRezultat, int &Dim)  
{  
    // Setarea dimensiunii matricii și vectorului  
    do {  
        <...>  
    } while (Dim <= 0);  
    // Alocare memorie  
    <...>  
    // Initializare valori pentru matrice și vector  
    InitializareSimpla(pMatrice, pVector, Dim);  
}
```

În continuare vom dezvolta două noi funcții care ne vor ajuta să controlăm datele de intrare. Aceste funcții, numite *PrintMatrice* și *PrintVector*, permit afișarea pe ecran a valorilor matricii și a vectorului. Argumentele funcției *PrintMatrice* sunt: matricea *pMatrice*, numărul de rânduri *RowCount* și numărul de coloane *ColCount*. Argumentele funcției *PrintVector* sunt: vectorul *pVector* și numărul de elemente *Dim*.

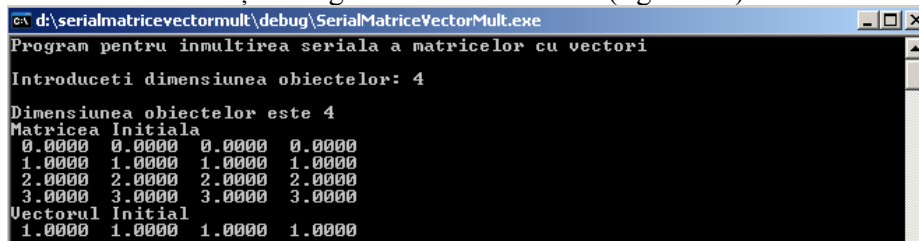
```
// Functie pt. afisare la consola a elementelor matricii  
void PrintMatrice (double* pMatrice, int RowCount, int  
ColCount) {  
    int i, j;  
    for (i=0; i<RowCount; i++) {  
        for (j=0; j<ColCount; j++)  
            printf("%7.4f ", pMatrice[i*RowCount+j]);  
        printf("\n");  
    }  
}  
// Functie pt. afisarea la consola a elementelor vectorului  
void PrintVector (double* pVector, int Dim) {  
    int i;  
    for (i=0; i<Dim; i++)  
        printf("%7.4f ", pVector[i]);  
    printf("\n");  
}
```

Cele 2 funcții vor fi apelate din funcția *main*:

```
<...>
// Alocare memorie si initializare
    InitializareProces(pMatrice, pVector, pRezultat, Dim);

// Afisare matrice si vector
printf ("Matricea Initiala \n");
PrintMatrice(pMatrice, Dim, Dim);
printf("Vectorul Initial \n");
PrintVector(pVector, Dim);
```

Compilați și rulați aplicația. Asigurați-vă că datele de intrare sunt setate în concordanță cu regulile descrise mai sus (figura 3.5).



```
d:\serialmatricevectormult\debug\SerialMatriceVectorMult.exe
Program pentru înmulțirea seriala a matricelor cu vectori
Introduceti dimensiunea obiectelor: 4
Dimensiunea obiectelor este 4
Matricea Initiala
0.0000 0.0000 0.0000 0.0000
1.0000 1.0000 1.0000 1.0000
2.0000 2.0000 2.0000 2.0000
3.0000 3.0000 3.0000 3.0000
Vectorul Initial
1.0000 1.0000 1.0000 1.0000
```

Figura 3.5 Rezultatul execuției programului după îndeplinirea pasului 3

Pas 4 – Terminarea execuției programului

Înainte de realizarea funcției de înmulțire a matricei cu vectorul vom dezvolta funcția ce permite terminarea corectă a aplicației. Pentru acest scop este necesar să eliberăm memoria ce a fost alocată în mod dinamic pe parcursul execuției programului. Astfel, vom construi funcția numită *FinalizareProces*. Argumentele funcției *FinalizareProces* sunt: matricea *pMatrice*, vectorul *pVector* și rezultatul înmulțirii *pRezultat*.

```
// Functie ce elibereaza memoria la incheierea procesului
de inmultire
void FinalizareProces(double* pMatrice, double*
pVector, double* pRezultat) {
    delete [] pMatrice;
    delete [] pVector;
    delete [] pRezultat;
}
```

Funcția *FinalizareProces* va trebui apelată la sfârșitul funcției *main*.

```
// Eliberare memorie
    FinalizareProces(pMatrice, pVector, pRezultat);
```


Pas 5 – Implementarea funcției de înmulțire matrice-vector

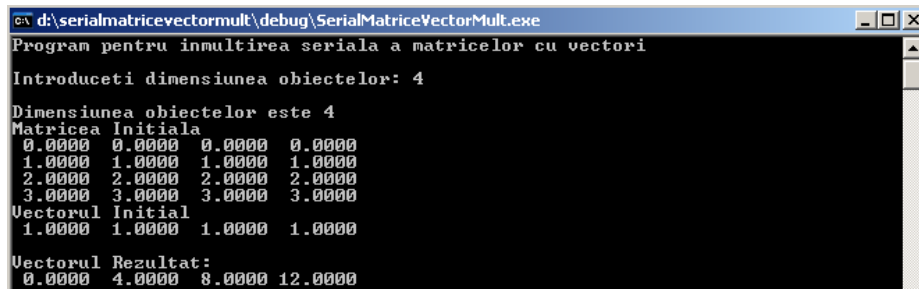
Funcția responsabilă de înmulțirea matricei *pMatrice* cu vectorul *pVector* se numește *CalculareRezultat* și reprezintă partea centrală a aplicației. Argumentele funcției *CalculareRezultat* sunt: matricea *pMatrice*, vectorul *pVector*, rezultatul înmulțirii *pRezultat* și dimensiunea *Dim*. În concordanță cu algoritmul prezentat în **Exercițiul 1**, codul funcției va fi următorul:

```
// Functie ce permite inmultire matrice-vector
void CalculareRezultat(double* pMatrice, double* pVector,
double* pRezultat, int Dim)
{
    int i, j;
    for (i=0; i<Dim; i++) {
        pRezultat[i] = 0;
        for (j=0; j<Dim; j++)
            pRezultat[i] += pMatrice[i*Dim+j]*pVector[j];
    }
}
```

Funcția *CalculareRezultat* va fi apelată din programul principal *main* și pentru verificarea corectitudinii operațiilor vom afișa pe ecran rezultatul înmulțirii:

```
void main()
{
    <...>
    // Start algoritm inmultire matrice-vector
    CalculareRezultat(pMatrice, pVector, pRezultat, Dim);
    // Afiare rezultat inmultire
    printf ("\nVectorul Rezultat: \n");
    PrintVector(pRezultat, Dim);
    <...>
}
```

Compilați și rulați aplicația. Analizați rezultatele obținute. Dacă programul este executat corect, rezultatul înmulțirii va trebui să aibă următoarea structură: elementul *i* din vectorul rezultat va fi egal cu produsul dintre dimensiunea vectorului și numărul de pe linia *i*. Astfel, dacă dimensiunea este 4, rezultatul înmulțirii va fi următorul: *pRezultat* = (0, 4, 8, 12) (figura 3.6).



```

c:\d:\serialmatricevectormult\debug\SerialMatriceVectorMult.exe
Program pentru înmulțirea seriala a matricelor cu vectori

Introduceti dimensiunea obiectelor: 4
Dimensiunea obiectelor este 4
Matricea Initiala
0.0000 0.0000 0.0000 0.0000
1.0000 1.0000 1.0000 1.0000
2.0000 2.0000 2.0000 2.0000
3.0000 3.0000 3.0000 3.0000
Vectorul Initial
1.0000 1.0000 1.0000 1.0000
Vectorul Rezultat:
0.0000 4.0000 8.0000 12.0000
    
```

Figura 3.6 Rezultatul operației de înmulțire matrice-vector

Pas 6 – Finalizarea experimentelor

Pentru a putea testa viteza de calcul *paralel* a operației de înmulțire dintre matrice și vector, la început este necesar să calculăm timpul necesar pentru execuția *secvențială* a algoritmului. Este util să testăm timpul de execuție pentru dimensiuni mari ale matricei și vectorului. Pentru aceasta vom genera aleatoriu elemente în matrice și vector prin intermediul funcției *InitializareAleatoare*. Argumentele funcției *InitializareAleatoare* sunt: matricea *pMatrice*, vectorul *pVector* și dimensiunea *Dim*.

```

// Functie ce permite initializarea aleatoare a elementelor
// matricii si vectorului
void InitializareAleatoare(double* pMatrice, double*
pVector, int Dim) {
    int i, j; // variabile necesare in bucla
    srand(unsigned(clock()));
    for (i=0; i<Dim; i++) {
        pVector[i] = rand()/double(1000);
        for (j=0; j<Dim; j++)
            pMatrice[i*Dim+j] = rand()/double(1000);
    }
}
    
```

În acest sens vom înlocui funcția *InitializareSimpla*, ce ne permitea să verificăm corectitudinea aplicației, cu funcția *InitializareAleatoare*.

```

void InitializareProces (double* &pMatrice, double*
&pVector, double* &pRezultat, int &Dim)
{
    <...>
    // Initializare valori pentru matrice si vector
    //InitializareSimpla(pMatrice, pVector, Dim);
    InitializareAleatoare(pMatrice, pVector, Dim);
}
    
```

Compilați și rulați aplicația. Asigurați-vă că datele sunt generate aleator. Pentru a determina timpul scurs pentru execuția programului sau pentru o parte din program se apelează o funcție numită: `time_t clock (void)`. Această funcție va returna timpul scurs între 2 apeluri succesive. Spre exemplu, fragmentul de cod de mai jos va calcula timpul scurs pentru execuția funcției `f()`.

```
time_t t1, t2 ;
t1 = clock();
f();
t2 = clock();
double durata = (t2 - t1)/double(CLOCKS_PER_SEC);
```

În continuare vom adăuga această funcție la aplicația noastră, înainte și după apelul funcției *CalculareRezultat*.

```
// Start algoritm inmultire matrice-vector
start = clock();
CalculareRezultat(pMatrice, pVector, pRezultat, Dim);
stop = clock();
durata = (stop-start)/double(CLOCKS_PER_SEC);
// Afiare rezultat inmultire
printf ("\nVectorul Rezultat: \n");
PrintVector(pRezultat, Dim);
// Afisare timp necesar pentru inmultire
printf("\nTimpul de executie: %f\n", durata);
```

Compilați și rulați aplicația. Pentru realizarea experimentelor pe diferite dimensiuni ale vectorului și matricei vom elimina porțiunea de cod ce afișează elementele acestora (comentând liniile de cod corespunzătoare). Rezultatele experimentelor vor fi înregistrate în tabelul 3.1.

Tabelul 3.1 *Rezultate experimentale înmulțire secvențială matrice-vector*

Număr Test	Dimensiune obiecte (matrice și vector)	Timp de execuție (sec.)
1	10	
2	100	
3	500	
4	1000	
5	2000	
6	3000	
7	4000	
8	5000	
9	6000	
10	7000	
11	8000	
12	9000	

În concordanță cu algoritmul de înmulțire matrice-vector prezentat în **Exercițiul 1**, rezultatul este un vector ce va avea dimensiunea Dim (se înmulțesc liniile matricei $pMatrice$ cu vectorul $pVector$). Astfel avem Dim operații pentru înmulțire linie matrice cu elemente vector și mai departe suma produselor obținute ($Dim-1$ operații). Numărul total de operații necesare pentru obținerea unui rezultat final este:

$$N = Dim \bullet (2 \bullet Dim - 1) \quad (3.2)$$

Pentru a putea estima timpul necesar pentru execuția paralelă a algoritmului, este necesar să cunoaștem durata τ pentru obținerea unui singur scalar. Astfel, pentru a evalua timpul necesar pentru execuția algoritmului, este necesar să înmulțim numărul de operații executate cu timpul necesar pentru una singură:

$$T_1 = N \bullet \tau = Dim \bullet (2 \bullet Dim - 1) \bullet \tau \quad (3.3)$$

În continuare vom prezenta un tabel în care se vor face comparații între timpul real necesar pentru execuția aplicației și timpul teoretic calculat conform formulei 3.3. Pentru a putea calcula timpul necesar pentru o singură operație vom aplica următoarea tehnică: alegem unul din experimente ca element central. Ca dovada, experimentul de înmulțire a matricei cu vector de dimensiune 5000 poate fi considerat ca element central. Execuția experimentului va fi divizibilă cu numărul de operații executate (numărul de operații poate fi calculat utilizând formula 3.2). Astfel, vom calcula timpul necesar pentru obținerea unui singur scalar. Apoi, utilizând aceasta valoare vom calcula timpul teoretic necesar pentru executarea experimentelor ramase. Rezultatele vor fi prezentate în tabelul 3.2.

Tabelul 3.2 Rezultate experimentale și teoretice pentru înmulțire secvențială matrice-vector

Timp τ (sec):			
Număr Test	Dimensiune obiecte (matrice și vector)	Timp de execuție (sec.)	Timp teoretic de execuție (sec)
1	10		
2	100		
3	500		
4	1000		
5	2000		
6	3000		
7	4000		
8	5000		
9	6000		
10	7000		
11	8000		
12	9000		

De notat este faptul că timpul necesar pentru obținerea unui singur scalar depinde, în general, de dimensiunea obiectelor implicate în operația de înmulțire. Această dependență poate fi explicată ținând cont de arhitectura calculatorului. Astfel, dacă obiectele sunt foarte mici, atunci acestea pot fi în întregime localizate în memoria cache a procesorului și timpul de acces pentru aceasta este foarte ridicat. Dacă algoritmul operează cu obiecte de dimensiuni medii ce pot fi în întregime localizate în RAM, dar nu în cache, timpul de execuție pentru o singură operație va fi mai mare, deoarece timpul de acces la memoria RAM este mai mare decât la cache. Dacă obiectele sunt suficient de mari și nu pot fi localizate în memoria RAM, atunci este apelat mecanismul de *swap*. În acest caz, obiectele sunt stocate pe un dispozitiv extern de memorare, iar timpul pentru scrierea și citirea memoriei crește semnificativ. Astfel, alegând un experiment ca pivot (experimentul pentru care este calculat timpul pentru obținerea unui singur scalar), va trebui să fim orientați spre operații cu valori medii. De aceea am ales ca pivot obiecte de dimensiune 5000.

Exercițiul 3 – Dezvoltarea algoritmului paralel de înmulțire matrice-vector

Principii de paralelizare

Dezvoltarea algoritmilor (în particular, metode de calcul paralel) paraleli pentru rezolvarea problemelor ingineresti complicate poate fi o adevărată provocare. În acest laborator plecăm de la premisa că problema de înmulțire a matricelor cu vectori este deja cunoscută. Activitățile pentru determinarea metodelor eficiente în calculul paralel sunt următoarele:

- ⇒ Analiza schemei de implementare a algoritmului și descompunerea acesteia în subtask-uri ce vor fi executate în paralel pe mai multe procesoare.
- ⇒ Analiza dependențelor între subtask-uri.
- ⇒ Determinarea sistemelor de calcul necesare pentru rezolvarea problemei și distribuirea subtask-urilor disponibile între procesoare.

Aceste etape de dezvoltare a algoritmilor paraleli au fost sugerate de I. Foster [2].

Repetarea operațiilor de calcul pentru diferite elemente ale matricei este tipică pentru toate metodele de înmulțire a matricelor cu vectori. Acest lucru demonstrează existența *paralelismului de date* în acest tip de aplicații. Ca rezultat, operațiile de paralelizare pot fi reduse, în cele mai multe cazuri, la distribuirea elementelor matricelor între procesoarele care alcătuiesc

sistemul de calcul. Existența diferitelor scheme de distribuție a datelor generează o serie de *algoritmi paraleli de înmulțire a matricelor cu vectori*. Cele mai generale și cele mai răspândite metode de partiționare a matricelor sunt:

1. Partiționare în benzi de linii sau coloane (*stripped matrix partitioning*). În acest caz, fiecărui procesor îi este asociat un anumit număr de rânduri din matrice (în cazul partiționării pe rânduri sau orizontale a matricei) sau un anumit număr de coloane din matrice (în cazul partiționării pe coloane sau verticale a matricei) – vezi figura 3.7a și 3.7b.

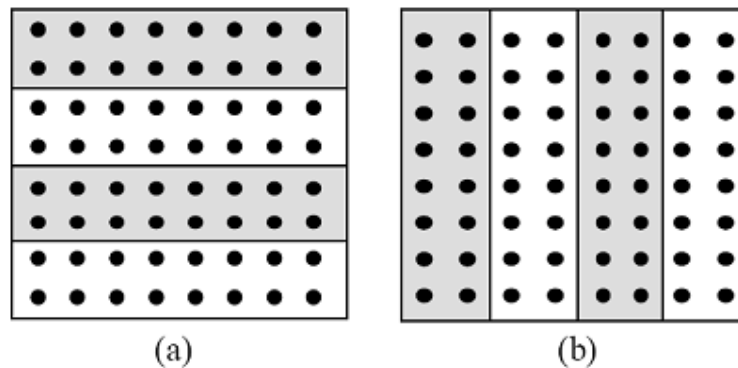


Figura 3.7 Modalități de distribuire a elementelor matricei între procesoare
(a) partiționare pe rânduri sau orizontală (b) partiționare pe coloane sau verticală

În cazul distribuirii matricei pe rânduri, matricea A poate fi reprezentată astfel: $A = (A_0, A_1, \dots, A_{p-1})^T$, $A_i = (a_{i_0}, a_{i_1}, \dots, a_{i_{k-1}})$, $i_j = ik + j$, $0 \leq j < k$,

$k = m / p$, unde $a_i = (a_{i1}, a_{i2}, \dots, a_{in})$, $0 \leq i \leq m$, reprezintă rândul i al matricei A (presupunem că numărul de rânduri m este divizibil cu numărul de procesoare p). Algoritmul paralel de înmulțire matrice-vector utilizat în acest laborator se bazează pe distribuirea matricei pe rânduri.

2. Partiționare în blocuri rectangulare sau pătrate (*chessboard matrix partitioning*). În acest caz, matricea este împărțită în blocuri dreptunghiulare. Dacă numărul de procesoare este $p=s*q$, numărul de rânduri ale matricei este divizibil cu s , numărul de coloane este divizibil cu q (spre ex. $m=k*s$ și $n=l*q$). Matricea A poate fi reprezentată ca un ansamblu de blocuri dreptunghiulare ca în figura 3.8.

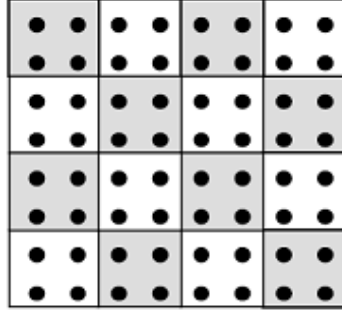


Figura 3.8 Modalități de distribuire a elementelor matricei între procesoare – partiționare în blocuri rectangulare sau pătrate

$$A = \begin{pmatrix} A_{00} & A_{02} & \dots & A_{0q-1} \\ & \dots & & \\ A_{s-11} & A_{s-12} & \dots & A_{s-1q-1} \end{pmatrix}, \text{ unde } A_{ij} \text{ este un bloc din matrice ce conține}$$

elementele:

$$A_{ij} = \begin{pmatrix} a_{i_0j_0} & a_{i_0j_1} & \dots & a_{i_0j_{l-1}} \\ & \dots & & \\ a_{i_{k-1}j_0} & a_{i_{k-1}j_1} & \dots & a_{i_{k-1}j_{l-1}} \end{pmatrix}, \text{ } i_v = ik+v, 0 \leq v < k, k=m/s, j_u=jl+u, 0 \leq u \leq l,$$

$$l=n/q.$$

În acest caz este convenabil ca sistemul de calcul să aibă fizic sau cel puțin logic o topologie de tip grid. Apoi, datele vor fi distribuite în mod continuu tuturor procesoarelor ce alcătuiesc structura de grid. În continuare vom prezenta algoritmul de înmulțire matrice-vector bazat pe împărțirea matricei pe rânduri (linii orizontale). În acest caz, operațiile de bază care se execută constau în înmulțirea scalară a rândurilor matricei cu vectorul.

Analiza dependențelor informaționale

Pentru a putea realiza *operația fundamentală* de înmulțire scalară, procesorul trebuie să conțină rândul corespunzător al matricei *pMatrice* și o copie a vectorului *pVector*. După terminarea calculului, fiecare *operație fundamentală* determină un element al rezultatului, al vectorului *pRezultat*.

Schema generală de interacțiune între subtask-uri în cursul execuției calculului este prezentată în figura 3.9.

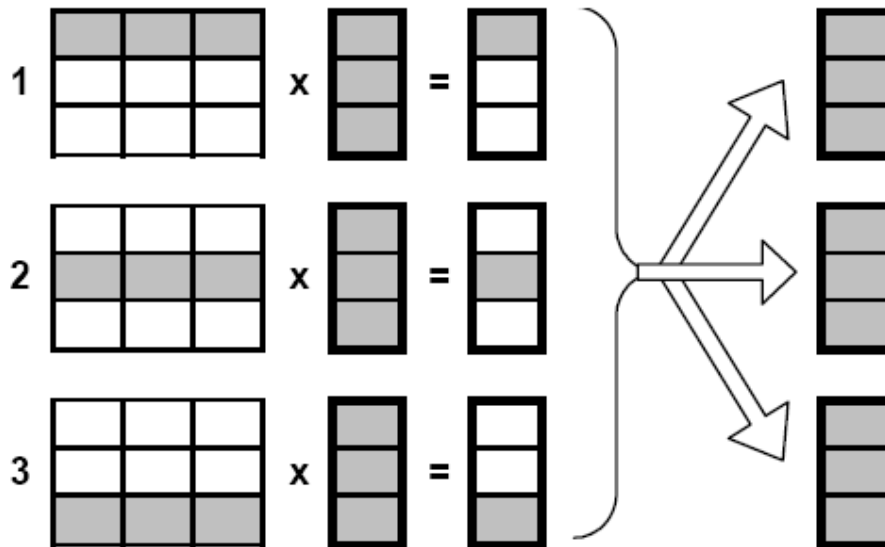


Figura 3.9 Organizarea calculului în cazul algoritmului paralel de înmulțire matrice-vector bazat pe distribuția matricii pe rânduri

Pentru a combina rezultatele obținute pe fiecare procesor în scopul obținerii vectorului *pRezultat*, este necesară execuția unor operații de tipul *all gather* prin intermediul cărora fiecare procesor transmite rezultatele sale către celelalte procesoare. Acest lucru poate fi realizat, spre exemplu, cu ajutorul funcției ***MPI_Allgather*** din librăria MPI (figura 3.10).

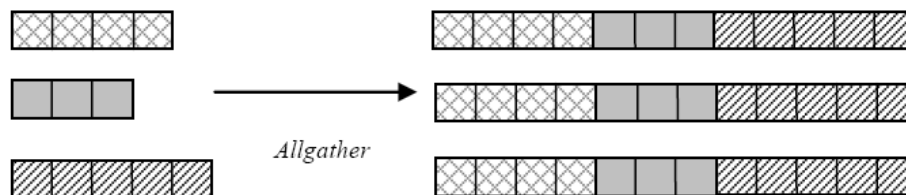


Figura 3.10 Comunicarea și schimbarea datelor între procesoare cu ajutorul funcției ***MPI_Allgather***

Distribuția subtask-urilor între procesoare

În procesul de înmulțire a matricii *pMatrice* cu vectorul *pVector*, numărul de operații fundamentale pentru obținerea produsului scalar este același pentru toate subtask-urile. De aceea, în cazul în care numărul de procesoare *p* este mai mic decât numărul de subtask-uri de bază *m* ($p < m$), putem combina aceste subtask-uri în așa fel încât fiecare procesor să execute câteva dintre acestea. Task-urile trebuie să corespundă unei secvențe

continue de rânduri ale matricei $pMatrice$. În acest caz, până la completarea calculului, fiecare subtask determină câteva elemente ale vectorului rezultat $pRezultat$. De asemenea, distribuirea subtask-urilor între procesoare se poate face și într-un mod arbitrar.

Exercițiul 4 – Codul paralel al programului ce permite înmulțirea matricelor cu vectori

Pentru realizarea sarcinilor propuse, vom dezvolta un program paralel de înmulțire matrice-vector. Pentru acest scop vor trebui realizate următoarele:

- Studierea librăriei MPI, structura programelor MPI și a funcțiilor de bază pe care această librărie le pune la dispoziție.
- Câștigarea experienței în dezvoltarea programelor paralele.

În cele ce urmează vom prezenta numai funcțiile MPI necesare pentru rezolvarea aplicației, o descriere generală a conceptului de MPI fiind tratat în detaliu în cadrul lucrărilor 1 (*Introducere în programare paralelă și distribuită. Standardul MPI.*) și 2 (*Programare paralelă și distribuită în MPI*).

Conceperea unui program paralel cu MPI

Un program paralel cu MPI se referă la startarea simultană a unui număr de procese. Procesele pot fi executate pe același procesor sau pe procesoare diferite. Așadar, simultan mai multe procese pot exista pe același procesor (în acest caz procesele sunt executate pe baza partajării timpului „time-shared mode”). În cazul cel mai defavorabil, un singur procesor poate fi utilizat pentru a executa toate procesele ce compun programul paralel. Ca o regulă, această metodă este utilizată pentru verificarea inițială a corectitudinii programului paralel. Numărul de procese și numărul de procesoare utilizate nu poate fi schimbat în timpul procesului de calcul în cazul standardului MPI-1 (standardul MPI-2 permite schimbarea dinamică a numărului de procesoare). Toate procesele ce alcătuiesc programul paralel sunt numerotate secvențial de la 0 la $p-1$, unde p este numărul total de procese.

MPI folosește obiecte numite *comunicatori* și *grupuri* pentru a defini ce colecții de procese pot comunica între ele. Cele mai multe funcții MPI necesită specificarea unui comunicator ca argument. Procesele ce alcătuiesc un program paralel sunt unite în *grupuri*. În MPI, *comunicatorul* este destinat unirii proceselor în același *grup*. Pentru simplitate, toate procesele din cadrul unui program paralel aparțin unui comunicator cu identificatorul

MPI_COMM_WORLD, ce este creat implicit. Ca regulă, operațiile de transmitere point-to-point a datelor este realizată între procese care aparțin aceluiași comunicator. Operațiile colective sunt aplicate simultan tuturor proceselor ce aparțin unui comunicator. În cursul derulării aplicației noi grupuri pot fi create, în vreme ce altele deja existente pot fi șterse. Același proces poate să aparțină la diferite grupuri și comunicatori.

Pas 1 – Deschiderea proiectului *ParalelMatriceVectorMult*

Se va deschide proiectul *ParalelMatriceVectorMult* parcurgând următorii pași:

- ⇒ Start Microsoft **Visual Studio 2005**.
- ⇒ Se execută comanda **Open** → **Project/Solution** în meniul **File**.
- ⇒ Se alege directorul *ParalelMatriceVectorMult* în fereastra **Open Project**
- ⇒ Se execută dublu click pe fișierul *ParalelMatriceVectorMult.sln*.

După ce s-a deschis proiectul, în fereastra **Solution Explorer** (Ctrl+Alt+L), se execută dublu click pe fișierul *ParalelMV.cpp*, ca în figura 3.11. Apoi, codul ce urmează a fi modificat și îmbunătățit, va fi deschis în fereastra **Visual Studio**.

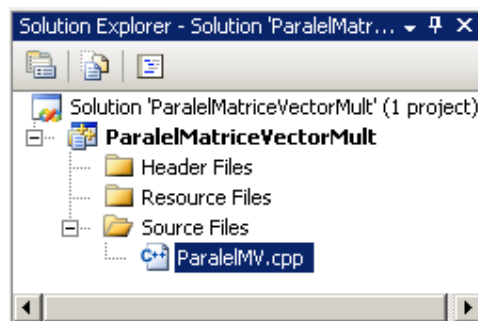


Figura 3.11 Deschiderea fișierului *ParalelMV.cpp*

Fișierul *ParalelMV.cpp* permite accesul la librăriile necesare și conține, de asemenea, variabilele necesare pentru dezvoltarea aplicației, precum și mesajele către utilizatori. Următoarele funcții vor fi copiate din proiectul anterior ce conține algoritmul serial de înmulțire matrice-vector: *InitializareSimpla*, *InitializareAleatoare*, *CalculareRezultat*, *PrintMatrice* și *PrintVector*. Aceste funcții au fost prezentate în detaliu în exercițiul 2 al acestui laborator. Funcțiile rămân valabile și pentru varianta paralelă a programului de înmulțire matrice-vector.

Pas 2 – Inițializarea și finalizarea programului paralel

Înainte de utilizarea funcțiilor din librăria MPI trebuie să includem header-ul MPI în aplicație. Pentru aplicația scrisă în C/C++, header-ul este numit *mpi.h*. Fișierul conține toate definițiile funcțiilor din librăria MPI. Adăugați linia de cod marcată la programul paralel de înmulțire matrice-vector.

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <conio.h>
#include <time.h>
```

Este necesar să inițializăm și să finalizăm în funcția main programul MPI. Pentru aceasta adăugați codul de mai jos imediat după declarația variabilelor.

```
void main(int argc, char* argv[])
{
    double* pMatrice; // Matricea initiala
    double* pVector; // Vectorul initial
    double* pRezultat; // Vectorul rezultat inmultire
    int Dim; // Dimensiunea matricii si a vectorului
    double start, stop, durata;

    MPI_Init(&argc, &argv);
    printf("Program pentru inmultirea paralela a matricelor
cu vectori\n");
    MPI_Finalize();
}
```

Dacă se va rula aplicația folosind spre exemplu 4 procese se vor obține rezultatele prezentate în figura 3.12.

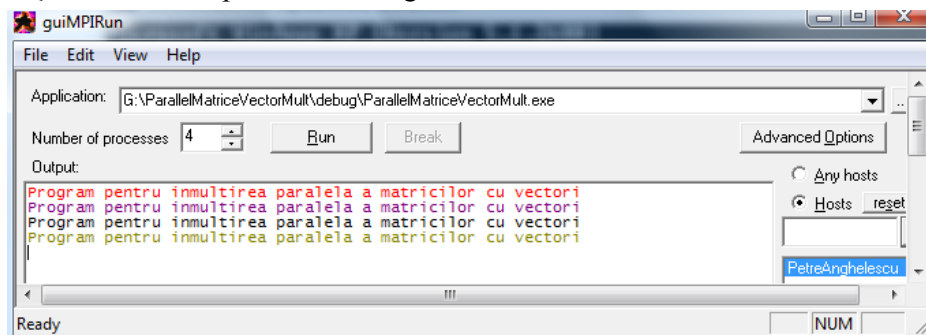


Figura 3.12 Rezultatul execuției programului paralel

Pas 3 – Determinarea numărului de procese

Numărul de procese într-un program paralel ce folosește librăria MPI se face prin utilizarea funcției `MPI_Comm_Size`. Dacă se va executa aplicația folosind 4 procese se va obține:

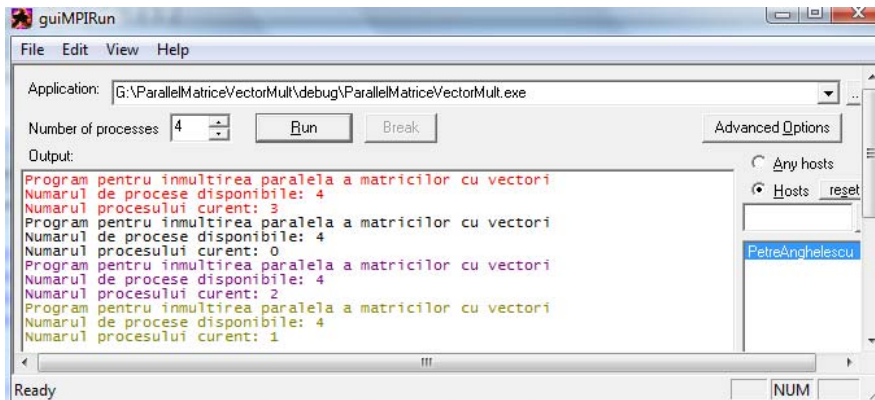


Figura 3.13 Determinare număr de procese

Se observă că fiecare proces în parte va afișa câte un mesaj. Este rezonabil să facem câteva schimbări în cod astfel încât mesajul inițial să fie afișat numai de un proces și anume de procesul cu numărul 0. Pentru aceasta se vor adăuga liniile de cod de mai jos.

```
if (ProcRank == 0) {  
    printf("Program pentru inmultirea paralela a matricilor cu  
    vectori\n");  
    printf("Numarul de procese disponibile: %d\n", ProcNum);  
}  
printf("Numarul procesului curent: %d\n", ProcRank);
```

Compilați și rulați din nou aplicația. Asigurați-vă că mesajul inițial și numărul de procese au fost afișate o singură dată de către procesul cu numărul 0, ca în figura 3.14.

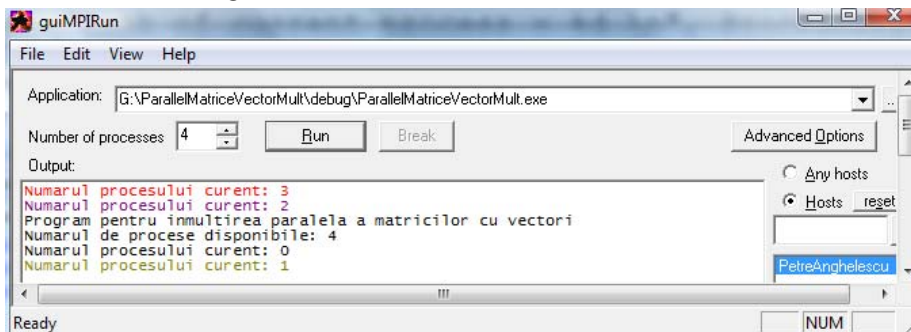


Figura 3.14 Afișare proces curent

Pas 4 – Citirea dimensiunii matricei și vectorului inițial

Funcția *InitializareProces* folosită în programul secvențial rămâne valabilă și în acest caz. În prima etapă este necesar să determinăm dimensiunea obiectelor (prin setarea valorii variabilei *Dim*). Dialogul cu utilizatorul pentru stabilirea dimensiunii obiectelor este realizat de un singur proces, și anume procesul cu rangul 0. Acest proces va fi în cele ce urmează numit *master* și va avea mereu rangul 0. Codul de mai jos va trebui adăugat în funcția *InitializareProces*:

```
// Functie ce alocă memoria necesară și initializează  
matricea și vectorul  
void InitializareProces (double* &pMatrice, double*  
&pVector, double* &pRezultat, int &Dim) {  
    // Setarea dimensiunii matricei și vectorului  
    printf("\nIntroduceți dimensiunea obiectelor: ");  
    scanf("%d", &Dim);  
    printf("\nDimensiunea obiectelor este %d\n", Dim);  
}
```

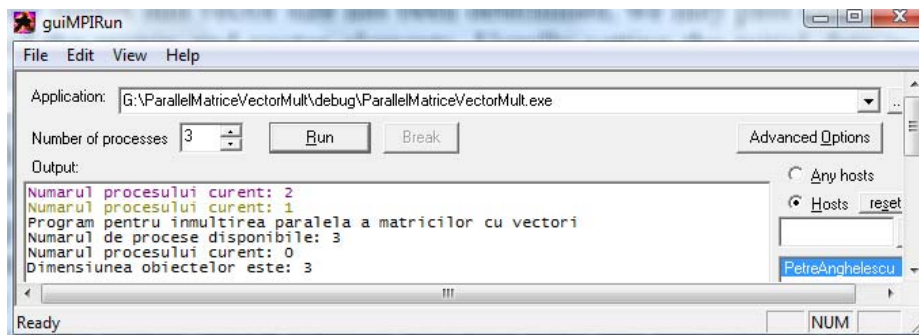


Figura 3.15 Afișare dimensiune matrice și vector

Pas 5 – Inițializarea cu date a matricei și a vectorului

După ce am stabilit dimensiunea obiectelor urmează alocarea dinamică a memoriei și stabilirea elementelor acestora.

```
double* pMatrice; // Matricea initiala  
double* pVector; // Vectorul initial  
double* pRezultat; // Vector rezultat inmultire  
int Dim=3; // Dimensiunea matricei si a vectorului initial  
double start, stop, durata;  
double* pProcRanduri; // Randuri matrice proces curent  
double* pProcRezultat; // Vectorul rezultat  
int NrRanduri; // Nr. de randuri din matrice ce  
revin procesului curent
```

În continuare vom modifica argumentele funcției *InitializareProces* astfel încât să determinăm numărul de rânduri *NrRanduri* din matrice ce vor fi distribuite fiecărui proces.

```
// Functie ce alocă memoria necesară și initializează  
matricea și vectorul  
void InitializareProces (double* &pMatrice, double*  
&pVector, double* &pRezultat, double* &pProcRanduri,  
double* &pProcRezultat, int &Dim, int &NrRanduri) {  
    if (ProcRank == 0) { <...> }  
    MPI_Bcast(&Dim, 1, MPI_INT, 0, MPI_COMM_WORLD);  
    //determinarea nr. randuri ale matricei ce vor fi  
    distribuite la fiecare proces  
    NrRanduri = Dim/ProcNum;  
  
    // Alocare dinamică memorie  
    pVector = new double [Dim];  
    pRezultat = new double [Dim];  
    pProcRanduri = new double [NrRanduri*Dim];  
    pProcRezultat = new double [NrRanduri];  
  
    // stabilirea elementelor matricii și vectorului  
    if (ProcRank == 0) {  
        pMatrice = new double [Dim*Dim]; // elem. matricei  
        // elementele vectorului  
        InitializareSimpla(pMatrice, pVector, Dim);  
    }  
}
```

Pentru afișarea elementelor matricei și vectorului se vor folosi funcțiile *pMatrice* și *pVector* realizate la programul serial de înmulțire matrice-vector. Rezultatele obținute pentru dimensiune obiecte (matrice, vector) 3 sunt prezentate în figura 3.16.

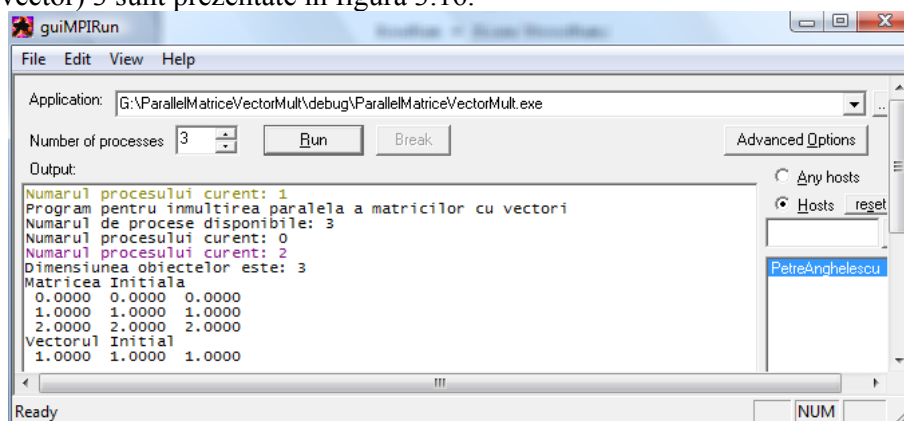


Figura 3.16 Afișare elemente matrice și vector

Pas 6 – Terminarea operațiilor de calcul

Înainte de realizarea funcției de înmulțire a matricei cu vectorul vom dezvolta funcția ce permite terminarea corectă a aplicației. Pentru acest scop este necesar să eliberăm memoria ce a fost alocată în mod dinamic pe parcursul execuției programului. Astfel, vom construi funcția numită *TerminareProces*. Argumentele funcției *TerminareProces* sunt: matricea *pMatrice*, vectorul *pVector*, rezultatul înmulțirii *pRezultat*, numărul de rânduri ale matricei ce vor fi memorate pe fiecare proces *pProcRanduri* și vectorul ce va fi memorat pe fiecare proces *pProcRezultat*. Funcția *TerminareProces* va trebui apelată la sfârșitul funcției *main*.

```
// eliberare memorie
void TerminareProces (double* pMatrice, double* pVector,
double* pRezultat, double* pProcRanduri, double*
pProcRezultat) {
    if (ProcRank == 0)
        delete [] pMatrice;
    delete [] pVector;
    delete [] pRezultat;
    delete [] pProcRanduri;
    delete [] pProcRezultat;
}
```

Compilați și rulați aplicația. Asigurați-vă că aplicația funcționează corect.

Pas 7 – Distribuirea datelor între procese

În concordanță cu schema de distribuție a datelor matricei, dată în exercițiul precedent, matricea trebuie distribuită pe linii orizontale egale și vectorul inițial trebuie copiat la toate procesele lansate în aplicație.

Funcția *DistributieDate* este responsabilă de acest lucru. Aceasta trebuie să aibă ca argumente matricea inițială *pMatrice*, vectorul *pVector*, rândurile matricei ce vor reveni fiecărui proces *pProcRanduri*, dimensiunea matricei și a vectorului *Dim* și numărul de rânduri din matrice ce revin fiecărui proces *NrRanduri*.

Pentru a copia vectorul inițial pe toate procesele, vom utiliza o comunicație de tip *broadcast* folosind funcția *MPI_Bcast*. Pentru a copia un anumit număr de rânduri din matrice la toate procesele vom folosi funcția *MPI_Scatter*. Cele două funcții (*MPI_Bcast* și *MPI_Scatter*) au fost

prezentate detaliat în laboratoarele precedente, motiv pentru care nu vom insista aici asupra sintaxei și modului de funcționare a acestora.

```
// distribuirea datelor intre procese
void DistributieDate(double* pMatrice, double*
pProcRanduri, double* pVector, int Dim, int NrRanduri)
{
    MPI_Bcast(pVector, Dim, MPI_DOUBLE, 0,
MPI_COMM_WORLD);
    MPI_Scatter(pMatrice, NrRanduri*Dim, MPI_DOUBLE, pProcRa
nduri, NrRanduri*Dim, MPI_DOUBLE, 0, MPI_COMM_WORLD);
}
```

Corespunzător, funcția *DistributieDate* va trebui apelată în *main* imediat după inițializare *InitializareProces*.

```
void main(int argc, char* argv[])
{
    <...>
    //Alocare memorie si initializare
    InitializareProces(pMatrice, pVector, pRezultat,
pProcRanduri, pProcRezultat, Dim, NrRanduri);

    // Distribuirea datelor intre procese
    DistributieDate(pMatrice, pProcRanduri, pVector, Dim,
NrRanduri);
    <...>
    MPI_Finalize();
}
```

Pentru a verifica corectitudinea datelor transmise între procese vom realiza o funcție de test numită *TestDistributie*.

```
void TestDistributie(double* pMatrice, double* pVector,
double* pProcRanduri, int Dim, int NrRanduri) {
    if (ProcRank == 0) { printf("Matricea Initiala: \n");
PrintMatrice(pMatrice, Dim, Dim);
printf("Vectorul Initial: \n");
PrintVector(pVector, Dim);
}
    MPI_Barrier(MPI_COMM_WORLD);
    for (int i=0; i<ProcNum; i++) {
        if (ProcRank == i) {
            printf("\nProc. Rank = %d \n", ProcRank);
            printf(" Elemente Matrice:\n");
            PrintMatrice(pProcRanduri, NrRanduri, Dim);
            printf(" Vector: \n");
            PrintVector(pVector, Dim);
        }
        MPI_Barrier(MPI_COMM_WORLD);
    }
}
```


Rezultatele obținute, pentru 3 procese și dimensiune obiecte (matrice, vector) 6, sunt prezentate în figura 3.17.

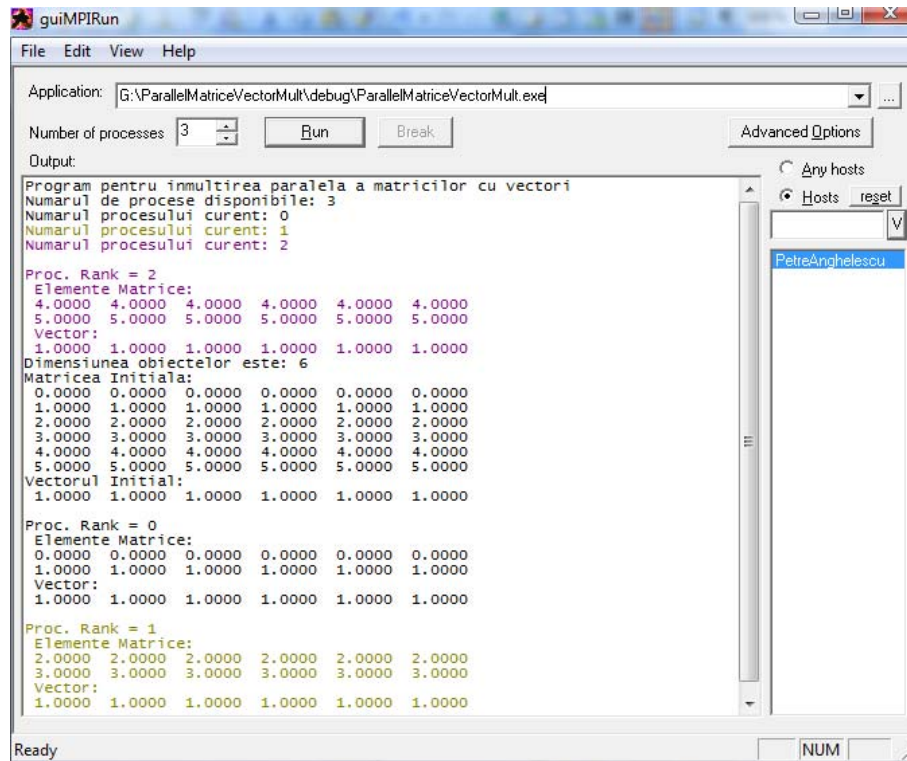


Figura 3.17 Distribuție date când aplicația utilizează 3 procese și dimensiunea obiectelor este 6

Pas 8 – Implementarea algoritmului paralel de înmulțire matrice-vector

Operația de înmulțire matrice-vector este realizată în funcția *CalculareParalelaRezultat*. Pentru a calcula un bloc din vectorul rezultat este necesar să avem acces la rândurile matricei *pProcRanduri*, vectorul inițial *pVector* și vectorul de pe fiecare proces *pProcRezultat*. De asemenea, trebuie cunoscută dimensiunea acestor obiecte. Astfel, vom construi funcția *CalculareParalelaRezultat* ce are următoarele argumente:

```
// inmulteste randurile matricei de pe fiecare procesor cu  
vectorul initial  
CalculareParalelaRezultat(pProcRanduri, pVector,  
pProcRezultat, Dim, NrRanduri);
```

Pentru a obține elemente din vectorul rezultat este necesar, similar ca la algoritmul secvențial, să înmulțim fiecare rând al matricei cu vectorul inițial. Diferența față de codul serial constă în faptul că procesul nu operează cu matricea inițială, ci cu *pProcRanduri* și procesează doar *NrRanduri* rânduri în loc de *Dim*. Funcția *CalculareParalelaRezultat* va fi apelată din *main* imediat după distribuirea datelor între procese.

```
// inmulteste matricea de pe fiecare procesor cu vectorul
initial
void CalculareParalelaRezultat(double* pProcRanduri,
double* pVector, double* pProcRezultat, int Dim, int
NrRanduri) {
    int i, j;
    for (i=0; i<NrRanduri; i++) {
        pProcRezultat[i] = 0;
        for (j=0; j<Dim; j++)
            pProcRezultat[i] += pProcRanduri[i*Dim+j]*
pVector[j];
    }
}
```

În continuare va trebui să testăm rezultatele parțiale obținute la fiecare proces. În acest sens vom construi o funcție numită *TestareRezultatePartiale*. Funcția *TestareRezultatePartiale* va fi apelată din *main* imediat după realizarea operației de înmulțire pe fiecare proces prin funcția *CalculareParalelaRezultat*.

```
// Functie pentru testarea partiala pe fiecare proces a
rezultatului inmultirii
void TestareRezultatePartiale(double* pProcRezultat, int
NrRanduri) {
    int i;
    for (i=0; i<ProcNum; i++) {
        if (ProcRank == i) {
            printf("\nProc. Rank = %d \n Vectorul partial
rezultat: \n", ProcRank);
            PrintVector(pProcRezultat, NrRanduri);
        }
        MPI_Barrier(MPI_COMM_WORLD);
    }
}
```

Rezultatele obținute, pentru 2 procese și dimensiune obiecte (matrice, vector) 6, sunt prezentate în figura 3.18. De exemplu, dacă aplicația paralelă utilizează 2 procese și obiectele (matricea și vectorul) au dimensiunea *Dim* egală cu 6, blocul de date (0, 6, 12) trebuie să apară la

primul proces, iar blocul (18, 24, 32) trebuie să apară la al doilea proces (figura 3.18).

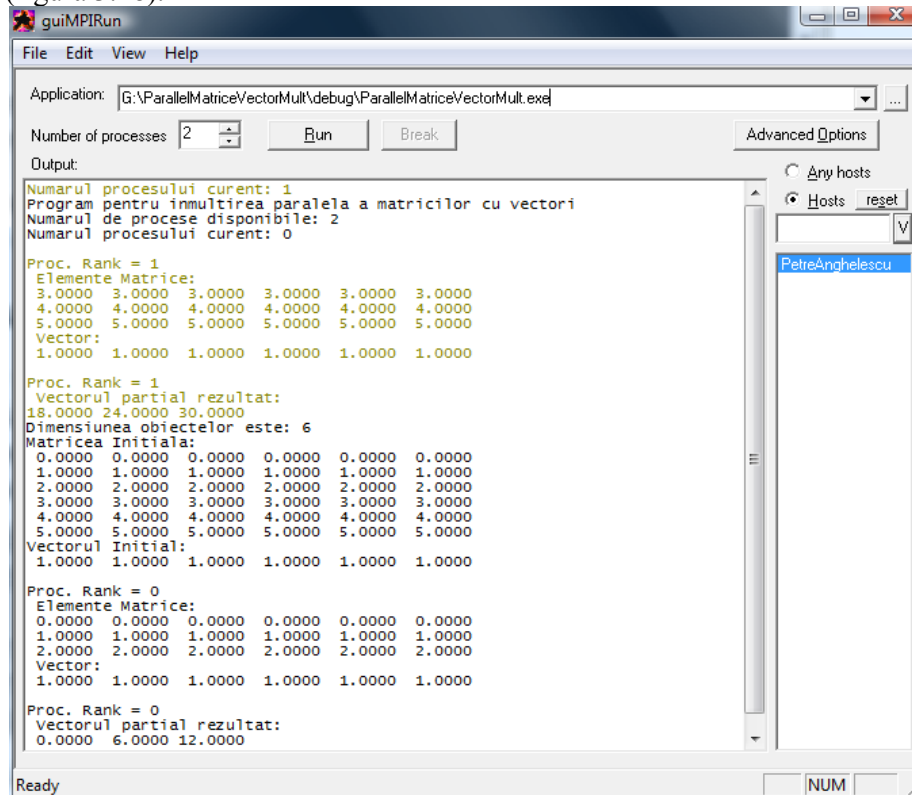


Figura 3.18 Rezultate parțiale obținute pe fiecare proces

Pas 9 – Colectarea rezultatelor parțiale de pe fiecare proces

Procedura de strângere a rezultatelor repeta distribuția inițială a datelor. Diferența constă în faptul că toate etapele trebuie executate în ordine inversă. Pentru a strânge datele de la procesoare și a forma rezultatul final vom folosi funcția *MPI_Allgather* a bibliotecii MPI. Aceasta funcție strânge datele de la toate procesele din comunicator într-un singur tablou și copiază acest tablou la toate procesele. Funcția *ReplicareRezultate* este responsabilă de strângerea rezultatelor parțiale.

```
void ReplicareRezultate(double* pProcRezultat, double*
pRezultat, int Dim, int NrRanduri) {
    MPI_Allgather(pProcRezultat, NrRanduri, MPI_DOUBLE,
pRezultat, NrRanduri, MPI_DOUBLE, MPI_COMM_WORLD); }
```

Funcția *ReplicareRezultate* va fi apelată din *main* imediat după realizarea operației de înmulțire pe fiecare proces prin funcția *CalculareParalelaRezultat*. După colectarea rezultatelor parțiale de la fiecare proces se va apela funcția *PrintVector* ce afișează rezultatul final (figura 3.19).

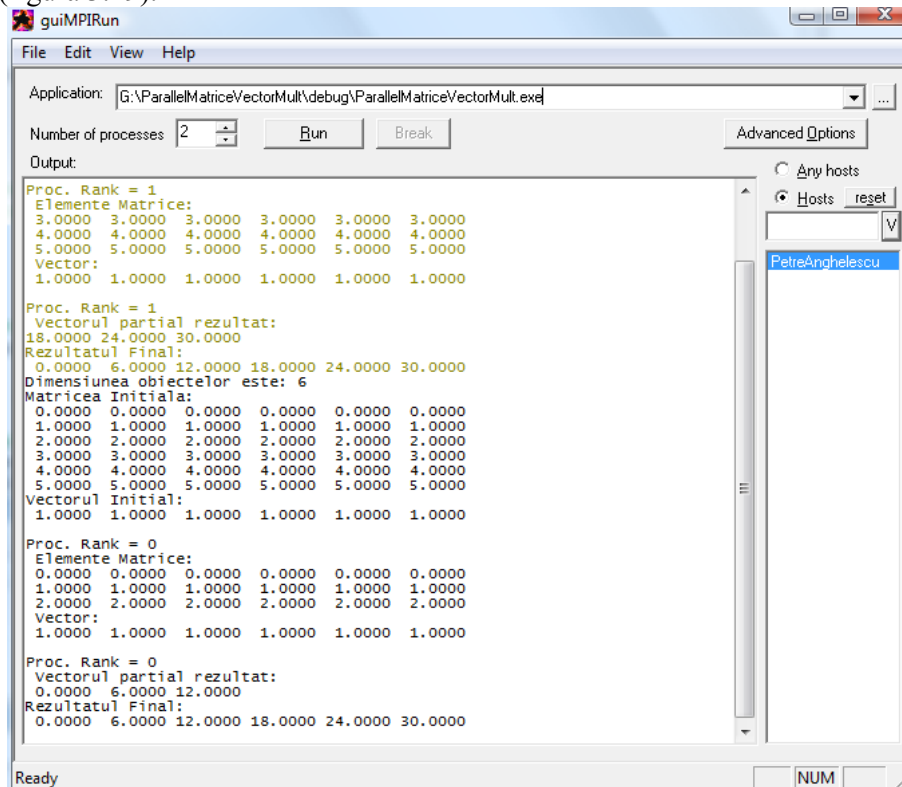


Figura 3.19 Rezultate finale obținute pe fiecare proces

Pas 10 – Finalizare algoritm paralel de înmulțire matrice-vector

Obiectivul principal al algoritmilor paraleli este rezolvarea problemelor complicate într-un timp mult mai mic decât algoritmi seriali prin utilizarea unui număr mai mare de procesoare. Timpul pentru execuția programului paralel se presupune a fi mai mic decât cel pentru programul serial. Pentru a înregistra timpul pentru execuția programului paralel vom folosi funcția *MPI_Wtime()*. Cum programul paralel include distribuirea datelor, calculul rezultatelor parțiale pe fiecare procesor și adunarea rezultatelor parțiale, cronometrul trebuie pornit imediat după apelul funcției *DistributieDate* și oprit imediat după execuția funcției *ReplicareRezultate*.

```
double start, stop, durata;  
<...>  
    start = MPI_Wtime();  
    // Distribuirea datelor intre procese  
    DistributieDate(pMatrice, pProcRanduri, pVector, Dim,  
    NrRanduri);  
  
    // inmulteste matricea de pe fiecare procesor cu vectorul  
    initial  
    CalculareParalelaRezultat(pProcRanduri, pVector,  
    pProcRezultat, Dim, NrRanduri);  
  
    // Strangerea rezultatelor partiale  
    ReplicareRezultate(pProcRezultat, pRezultat, Dim,  
    NrRanduri);  
  
    stop = MPI_Wtime();  
    durata = stop-start;  
    if(ProcRank==0)  
        printf("timpul de lucru pt algoritmul paralel  
    de inmultire matrice vector este: %f\n", durata);
```

Rezultatele obținute sunt prezentate în figura 3.20.

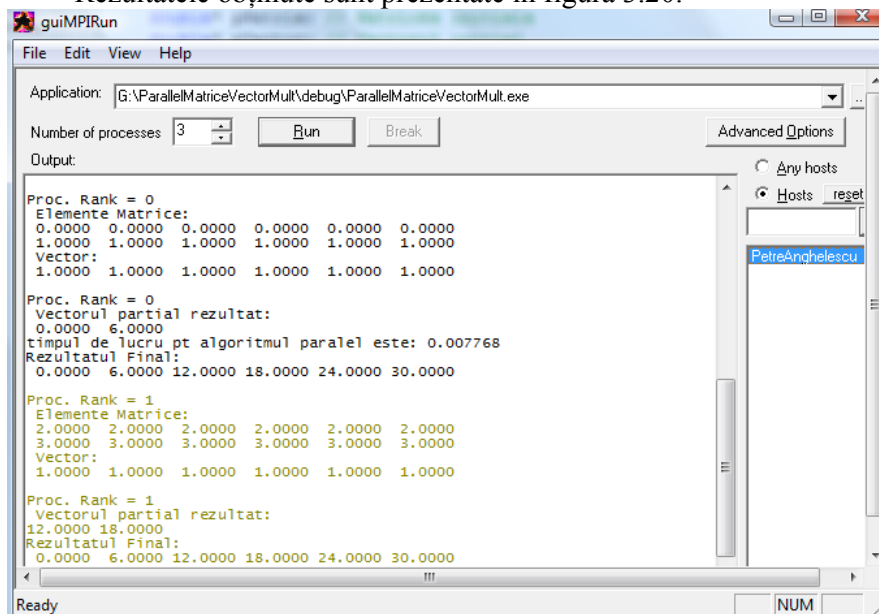


Figura 3.20 Afișare timp de lucru pentru algoritmul paralel de înmulțire matrice-vector

Compilați și rulați aplicația. Pentru realizarea experimentelor pe diferite dimensiuni ale vectorului și matricei vom elimina porțiunea de cod

ce afișează elementele acestora (comentând liniile de cod corespunzătoare). Rezultatele experimentelor vor fi înregistrate în tabelul 3.3.

Tabelul 3.3 Rezultate experimentale înmulțire paralelă matrice-vector

Dimensiune obiecte	Timp algoritm serial (sec)	Algoritm paralel			
		2 procesoare		4 procesoare	
		Timp (sec)	Accelerare	Timp (sec)	Accelerare
10					
100					
500					
1000					
2000					
3000					
4000					
5000					
6000					
7000					
8000					
9000					

În coloana a doua se introduce timpul obținut pentru algoritmul serial de înmulțire matrice-vector. Acest timp trebuie măsurat în cursul testării algoritmului secvențial de la **Exercițiul 2**. Pentru a calcula accelerația, se împarte timpul obținut la execuția serială la timpul obținut la execuția paralelă a algoritmului.

Pentru a putea estima timpul necesar pentru execuția paralelă a algoritmului dezvoltat potrivit schemei prezentate în **Exercițiul 3**, vom utiliza următoarea formulă:

$$T_p = \lceil n/p \rceil \cdot (2n-1) \cdot \tau + \alpha \lceil \log_2 p \rceil + w \lceil n/p \rceil (2^{\lceil \log_2 p \rceil} - 1) / \beta, \quad (3.4)$$

unde n este dimensiunea matricei și a vectorului, p este numărul de procesoare, τ este timpul de execuție pentru obținerea unui singur scalar (această valoare a fost calculată în cursul testării algoritmului secvențial), α este latența, iar β este lărgimea de bandă a rețelei de comunicație. În relația (3.4) operația $\lfloor \rfloor$ se referă la rotunjirea valorii obținute la cel mai apropiat număr întreg inferior, iar operația $\lceil \rceil$ se referă la rotunjirea valorii obținute la cel mai apropiat număr întreg superior.

În continuare, prin intermediul formulei 3.4, se va calcula timpul teoretic de execuție a algoritmului paralel. Rezultatele obținute vor fi trecute tabelul 3.4.

Tabelul 3.4 *Comparație rezultate experimentale-rezultate teoretice*

Nr. test	Dimensiune obiecte	Algoritm paralel			
		2 procesoare		4 procesoare	
		Model	Experiment	Model	Experiment
1	10				
2	100				
3	500				
4	1000				
5	2000				
6	3000				
7	4000				
8	5000				
9	6000				
10	7000				
11	8000				
12	9000				

Desfășurarea lucrării:

1. Se va citi breviarul teoretic. Se atrage atenția asupra faptului că toate cunoștințele din această lucrare vor fi necesare și în derularea celorlalte lucrări.
2. Se vor studia algoritmi prezentați pe parcursul acestui laborator urmărind efectul rulării acestora și buna lor execuție.
3. Se vor completa tabelele prezentate în lucrare cu datele experimentale/estimate obținute/calculate. Se vor comenta rezultatele obținute.
4. Se va studia algoritmul paralel de înmulțire matrice-vector bazat pe partiționarea matricei pe coloane. Se va realiza un program paralel ce implementează acest algoritm.
5. Se va studia algoritmul paralel de înmulțire matrice-vector bazat pe partiționarea matricei în blocuri rectangulare sau pătrate. Se va realiza un program paralel ce implementează acest algoritm.